Quantitative Specification of the Multi-Strategy Trading Bot

January 2026

Contents

Chapter 1

Introduction

This document presents a quantitative specification of the multi-strategy trading bot implemented in the repository. The focus is on the mathematical foundations of each strategy, the statistical validation procedures, and the risk management metrics that underpin production deployment. Frontend considerations are intentionally excluded.

1.1 Scope

The bot combines equity and options-based signals across multiple horizons:

- Market-neutral mean reversion on cointegrated equities and their option overlays.
- Trend-following breakouts via Donchian channels.
- A macro lead–lag framework that exploits cross-asset propagation delays.
- Volatility arbitrage through option Greeks and variance risk premia.
- A sector rotation model grounded in relative momentum and regime filters.

1.2 Data Sources and Assumptions

Market data is fetched through the `DataHandler` abstraction, which exposes historical bars and latest quotes. Unless otherwise specified, prices refer to adjusted closing levels. Options data is enriched with implied volatility and Greeks using a Black–Scholes evaluator. All monetary values are in USD.

1.3 Notation Overview

Mathematical symbols follow the conventions summarised in Appendix A. Random variables are uppercase (e.g., S_t for the underlying price), realised observations are lowercase, and estimated parameters include hats (e.g., $\hat{\sigma}$).

Chapter 2

System Architecture

The trading bot coordinates data ingestion, signal generation, execution, and risk monitoring through modular components orchestrated by the `TradingEngine`. The following code snippet highlights the bootstrapping sequence defined in `src/engine.py`.

```
# src/engine.py (abridged)
self.dh = DataHandler(paper_trading=True)
self.eh = ExecutionHandler(paper_trading=True)
self.pm = PortfolioManager(self.dh)
self.sm = StrategyManager(self.dh, self.eh, self.pm)
self.option_data_handler = OptionDataHandler(self.dh)
self.multi_leg_helper = MultiLegExecutionHelper(self.eh)
self.universe_manager = UniverseManager(...)
self.rebalance_strategies()
self.sm.reconcile_positions()
```

2.1 Data Layer

Historical and intraday bars are sourced through the Alpaca `StockHistoricalDataClient` inside the `DataHandler` (see `src/data_handler.py`). The mapping from textual timeframes ("1H", "1D", etc.) to `alpaca-py` objects is strict, ensuring deterministic sampling windows. For a symbol s and timeframe Δt, the handler returns a frame $\{(t_i, o_i, h_i, l_i, c_i, v_i)\}_{i=1}^{N}$ with timestamps aligned to UTC.

2.2 Execution Layer

The `ExecutionHandler` (`src/execution.py`) translates abstract signals into Alpaca order requests. Market and limit orders are supported through

$$\text{Order}(s, q, \text{side}, \text{type}, \tau), \tag{2.1}$$

where s denotes the traded symbol, q the quantity, side $\in \{\text{buy}, \text{sell}\}$, type $\in \{\text{market}, \text{limit}\}$, and τ the time-in-force instruction.

2.3 Portfolio Governance

The `PortfolioManager` (`src/portfolio_manager.py`) enforces per-strategy budget limits. Each strategy k receives an equity allocation A_k with leverage L_k. A trade proposal with estimated cost C_k and current exposure E_k is approved if

$$E_k + C_k \leq A_k L_k. \tag{2.2}$$

Live equity is refreshed through brokerage API calls with a fallback cache when unavailable.

2.4 Strategy Orchestration

The `StrategyManager` (`src/strategy_manager.py`) maintains the active strategy set $\mathcal{S}$. At each heartbeat the manager:

1. Queries the `RegimeDetector` for the prevailing volatility regime R_t.
2. Skips strategies whose profiles conflict with R_t (e.g., suppress pairs trading under high volatility).
3. Invokes `generate_signal()` on each remaining strategy and forwards approved orders to execution.

2.5 Universe Management

Candidate assets are generated by the `UniverseManager`, which screens the symbol list for statistical features (e.g., cointegration for pairs trading). The resulting watchlist modulates which strategy configurations are spawned during `rebalance_strategies()`.

Chapter 3

Equity Strategies

This chapter formalises the equity-driven strategies implemented in `src/strategies`. Each subsection states the signal equations, execution logic, and representative parameter choices drawn from the production configuration.

3.1 Cointegrated Pairs Mean Reversion

The `PairsTradeStrategy` maintains a market-neutral spread between equities S_t^A and S_t^B with hedge ratio $\hat{\beta}$ computed offline. The spread process is

$$X_t = S_t^A - \hat{\beta} S_t^B. \tag{3.1}$$

Assuming (S_t^A, S_t^B) are cointegrated, X_t is stationary with mean μ and standard deviation σ. Online statistics are re-estimated over a rolling window $\mathcal{W}$ via

$$\hat{\mu}_t = \frac{1}{|\mathcal{W}|} \sum_{u \in \mathcal{W}} X_u, \quad \hat{\sigma}_t = \sqrt{\frac{1}{|\mathcal{W}|} \sum_{u \in \mathcal{W}} (X_u - \hat{\mu}_t)^2}. \tag{3.2}$$

A z-score normalisation triggers trades:

$$Z_t = \frac{X_t - \hat{\mu}_t}{\hat{\sigma}_t}. \tag{3.3}$$

With entry threshold θ_{in} and exit threshold θ_{out}, orders follow

$$\begin{aligned} Z_t \geq \theta_{\text{in}} &\Rightarrow \text{short spread (sell } S^A\text{, buy } \hat{\beta} \text{ shares of } S^B), \\ Z_t \leq -\theta_{\text{in}} &\Rightarrow \text{long spread (buy } S^A\text{, sell } \hat{\beta} \text{ shares of } S^B), \\ |Z_t| \leq \theta_{\text{out}} &\Rightarrow \text{close outstanding position.} \end{aligned}$$

Example. For symbols "GS" and "MS" with $\hat{\beta} = 0.84$, $\theta_{\text{in}} = 1.25$, $\theta_{\text{out}} = 0.25$, and position size $q = 10$, observing $Z_t = 1.40$ generates orders $(-q, +q\hat{\beta}) = (-10, +8)$.

Implementation. This code snippet shows the core computation in `src/strategies/pairs_trade.py`.

```
spread = price_a - self.hedge_ratio * price_b
z_score = (spread - self.spread_mean) / self.spread_std
signals = self._evaluate_rules(z_score)
```

3.2 Donchian Channel Breakout

The `DonchianBreakoutStrategy` trades momentum breakouts using the Donchian envelope defined by the rolling extrema of closing prices C_t over lookback n:

$$U_t = \max_{u \in [t-n, t-1]} C_u, \qquad L_t = \min_{u \in [t-n, t-1]} C_u. \tag{3.4}$$

Signals are

$$\begin{aligned} C_t > U_t &\Rightarrow \text{enter long}, \\ C_t < L_t &\Rightarrow \text{enter short}, \\ \text{otherwise} &\Rightarrow \text{hold position}. \end{aligned}$$

Positions persist until a counter-signal occurs.

Example. With $n = 20$ and quantity $q = 10$, if C_t closes 1.2% above U_t, the strategy buys q shares; a drop below L_t later flips to a short.

Implementation. The vectorised backtest multiplies the signalled stance with forward returns to obtain trade-level performance for validation routines.

```
upper = df['close'].rolling(lookback).max().shift(1)
lower = df['close'].rolling(lookback).min().shift(1)
signals[df['close'] > upper] = 1
signals[df['close'] < lower] = -1
strategy_returns = signals * market_returns
```

3.3 Macro Lead–Lag Arbitrage

The `MacroArbitrageStrategy` exploits delayed transmission from a leader asset L to a laggard G. Over a lookback window of m minutes the leader return is

$$R^L_{t,m} = \frac{P^L_t}{P^L_{t-m}} - 1, \tag{3.5}$$

while the laggard move is constrained by $|R^G_{t,m}| < \delta$ to ensure it has not reacted. A z-score computed on the leader–laggard spread,

$$Z_t = \frac{R^L_{t,m} - \mathbb{E}[R^L_{t,m}]}{\sqrt{\mathrm{Var}(R^L_{t,m})}}, \tag{3.6}$$

triggers trades when $|Z_t| > \theta$. Qualifying signals are filtered by natural language sentiment scores σ_t derived from partner news feeds to suppress idiosyncratic jumps.

Execution. Upon a positive shock in the leader ($Z_t > \theta$), the bot shorts the leader and buys the laggard, sized to equal dollar exposure. Positions are held for h minutes (default $h = 120$) or until the spread normalises.

3.4 Sector Momentum Rotation

The `SectorMomentumStrategy` implements monthly relative-strength rotation among the SPDR sector universe $\mathcal{U}$. For each candidate $s \in \mathcal{U}$ the six-month momentum score is

$$M_s = \frac{C_s^{(t)}}{C_s^{(t-126)}} - 1. \tag{3.7}$$

The top N sectors by M_s are retained, subject to a regime filter that requires the closing price to exceed its 200-day simple moving average ($C_s^{(t)} > \mathrm{SMA}_{200,s}(t)$). Assets failing the filter are replaced with the defensive Treasury ETF D.

Allocation. Capital A is split equally across the final basket $\mathcal{B}$, yielding target weights $w_s = 1/|\mathcal{B}|$. The rebalance transacts $\Delta q_s = (w_s A / C_s^{(t)}) - q_s^{\mathrm{prev}}$ shares.

Example. If the top momentum names are XLK, XLY, and XLF, but XLY trades below its SMA_{200}, the portfolio holds {XLK, SHV, XLF} with one-third allocation each.

Chapter 4

Option Strategies and Pricing Framework

Option-based modules now rely on a Heston stochastic volatility engine backed by a Longstaff–Schwartz Monte Carlo (LSM) solver (`src/options/greeks.py`). The previous Black–Scholes analytic stack could not value American-style contracts nor capture volatility smiles. This chapter summarises the migration to the Heston model, details the batching improvements that keep Monte Carlo practical inside the bot, and revisits the option pair and volatility arbitrage overlays.

4.1 From Black–Scholes to Heston

The legacy analytics assumed log-normal diffusion and relied on closed-form Greeks. For short-dated equity options the approximation was acceptable, but portfolio risk measurements broke down for early-exercise products and instruments with pronounced smiles/skews. The new engine models the joint dynamics

$$\mathrm{d}S_t = \mu S_t \,\mathrm{d}t + \sqrt{v_t} S_t \,\mathrm{d}W_t^{(1)}, \tag{4.1}$$

$$\mathrm{d}v_t = \kappa(\theta - v_t)\,\mathrm{d}t + \sigma\sqrt{v_t}\,\mathrm{d}W_t^{(2)}, \tag{4.2}$$

$$\rho = \operatorname{corr}\big(W_t^{(1)}, W_t^{(2)}\big), \tag{4.3}$$

where v_t is the instantaneous variance. American prices are produced by simulating N risk-neutral paths with Euler discretisation, then applying the LSM regression to estimate continuation values. Greeks are recovered by finite differences but, crucially, the Monte Carlo paths are reused across bumps to reduce estimator variance.

4.1.1 Implementation Highlights

- **Reusable path factors.** The pricer first generates unit-normalised spot paths $\tilde{S}_{t_j}^{(n)}$ under the Heston dynamics. Pricing a contract with spot S_0 simply scales these factors by S_0. Delta and gamma perturbations reuse the same path bundle, guaranteeing smooth finite differences.
- **Batch pricing API.** The new `price_series` helper down-samples long spot histories, evaluates a single path bundle per scenario, and shares it across all strike/volatility bumps. This keeps the bootstrap process for spread statistics below a few hundred milliseconds.
- **Parameter overrides.** Strategy configurations may pin mean reversion (κ), long-run variance (θ), or initial variance v_0 when calibrated surfaces are available. Otherwise they default to squaring the supplied implied volatility.

This code snippet captures the core batching loop inside the greeks module.

```
path_factors = self._generate_path_factors(ttm, rate, params, seed)
price = self._price(opt_type, spot, strike, ttm, rate, params, seed, path_factors)

bump = max(self.greek_bumps.get("spot", 0.01) * spot, 1e-4)
up_price = self._price(opt_type, spot + bump, strike, ttm, rate, params, seed, path_factors)
down_price = self._price(opt_type, max(spot - bump, 1e-4), strike, ttm, rate, params, seed, path_factors)
delta = (up_price - down_price) / (2 * bump)

vega_path_up = self._generate_path_factors(ttm, rate, up_params, seed)
vega_path_dn = self._generate_path_factors(ttm, rate, dn_params, seed)
vega = (
    self._price(opt_type, spot, strike, ttm, rate, up_params, seed, vega_path_up)
    - self._price(opt_type, spot, strike, ttm, rate, dn_params, seed, vega_path_dn)
) / (up_vol - dn_vol)
```

4.2 Option Pair Mean Reversion

The `OptionPairsStrategy` extends equity pairs trading by operating on option premiums. For symbols S^A and S^B, at-the-money options with target delta δ^* are selected by solving

$$\Delta(S_t^i, K_t^i, \sigma_t^i, \tau) = \delta^*, \qquad i \in \{A, B\}, \tag{4.4}$$

where σ_t^i is the historical volatility estimate returned by `OptionDataHandler`. Delta here is the Monte Carlo finite difference described earlier, reusing the already simulated paths. The option spread is

$$Y_t = P_t^A - \kappa_t P_t^B, \tag{4.5}$$

with vega ratio $\kappa_t = \max(0.1, {}^A_t / \max(|{}^B_t|, 10^{-6}))$ enforcing vega-neutral hedging. Z-score evaluation mirrors the equity variant:

$$Z_t = \frac{Y_t - \bar{Y}}{s_Y}. \tag{4.6}$$

Trading rules choose between shorting expensive volatility (sell P^A, buy κ_t contracts of P^B) or the inverse.

Bootstrapping efficiency. Historical spread statistics formerly required thousands of Black–Scholes evaluations. With the batchable Heston engine the bot simulates one path bundle per strike pair, dramatically reducing startup time while still capturing early-exercise optionality in the sampled distribution.

Example. Suppose $Y_t = 1.80$ dollars, $\bar{Y} = 1.10$, $s_Y = 0.40$, yielding $Z_t = 1.75$ against thresholds $\theta_{\text{in}} = 1.5$ and $\theta_{\text{out}} = 0.0$. The strategy enters a short spread: sell one contract of P^A and buy $\lceil \kappa_t \rceil$ contracts of P^B.

4.3 Volatility Arbitrage (Gamma Scalping)

The `VolatilityArbitrageStrategy` targets the variance risk premium by shorting implied volatility when its ratio to realised volatility exceeds a threshold. Realised volatility is measured via the Parkinson estimator on high–low ranges:

$$\widehat{\sigma}_{\text{RV}} = \sqrt{\frac{1}{4n \ln 2} \sum_{u=1}^{n} \ln^2\!\left(\frac{H_u}{L_u}\right)} \times \sqrt{252}. \tag{4.7}$$

Implied volatility σ_{IV} is extracted from option chains near 30 days to expiry. The entry condition is

$$\frac{\sigma_{\text{IV}}}{\widehat{\sigma}_{\text{RV}}} > \theta_{\text{IV}}, \tag{4.8}$$

with default $\theta_{\text{IV}} = 1.25$. The strategy sells an at-the-money straddle (or iron condor variant) and delta-hedges dynamically: when the underlying delta drifts beyond $\pm\theta_{\Delta}$ (25 deltas by default), equity shares are traded to re-centre the position, capturing gamma scalping gains.

Risk Controls. Positions are closed on (i) premium decay hitting the profit target fraction π (e.g., 50%), or (ii) implied volatility expansion to $\theta_{\text{SL}}\,\sigma_{\text{IV,entry}}$ with $\theta_{\text{SL}} = 1.3$. The Heston engine provides more realistic vega and gamma for these stop checks, especially when the skew steepens during stress.

Example. Consider SPY options with $\sigma_{\text{IV}} = 0.28$, $\widehat{\sigma}_{\text{RV}} = 0.20$. The ratio 1.40 exceeds 1.25, prompting the sale of one straddle. If underlying delta drifts to +32 the bot sells 0.32 delta-equivalent shares (rounded) to remain neutral.

Chapter 5

Backtesting Framework

Historical simulations evaluate each strategy on aligned price series obtained from the `DataHandler`. This chapter summarises the mechanics common across `run_backtest` implementations.

5.1 Data Alignment and Rolling Windows

For multi-asset strategies a merged dataframe $\mathbf{P}$ is built by left-joining individual close series and dropping rows with missing values. Rolling statistics use window length w derived from the requested timeframe Δt and lookback days D:

$$w = \max\left(10, \left\lfloor \frac{D \times 1440}{\text{minutes}(\Delta t)} \right\rfloor\right). \tag{5.1}$$

5.2 Equity Pairs Simulation

The pairs backtest iterates chronologically, recomputing the spread and z-score per bar, entering and exiting positions with transaction size q. Realised P&L per completed trade j is

$$\Pi_j = q(P^A_{\text{exit}} - P^A_{\text{entry}}) - q\hat{\beta}(P^B_{\text{exit}} - P^B_{\text{entry}}). \tag{5.2}$$

Cumulative equity E_t evolves via

$$E_{t+1} = E_t + \Pi_t. \tag{5.3}$$

This snippet abridges the reporting logic from `src/strategies/pairs_trade.py`.

```
189 prices['spread'] = prices[self.symbol_a] - self.hedge_ratio *
        prices[self.symbol_b]
190 prices['spread_mean'] = prices['spread'].rolling(window=window).
        mean()
```

```
prices['spread_std'] = prices['spread'].rolling(window=window).std
    (ddof=0)
prices['z_score'] = (prices['spread'] - prices['spread_mean']) /
    prices['spread_std']
```

5.3 Option Spread Simulation

The option pairs backtest reconstructs synthetic option prices using the Black–Scholes greeks and estimated volatility series. After generating the spread Y_t and vega ratio, returns are computed as percentage moves of the option legs, which feed into cumulative equity with contract notional $\mathcal{N}$.

5.4 Performance Metrics

Across strategies the following statistics are reported:

$$\text{Total P\&L} = \sum_j \Pi_j, \qquad \text{Win Rate} = \frac{\#\{\Pi_j > 0\}}{\#\{j\}}, \tag{5.4}$$

$$\text{Sharpe} = \frac{\mathbb{E}[R]}{\sigma_R}\sqrt{\text{PeriodsPerYear}}, \quad \text{Max Drawdown} = \max_t \left(1 - \frac{E_t}{\max_{u \le t} E_u}\right). \tag{5.5}$$

Benchmarked strategies also track excess return relative to a reference index.

Chapter 6

Validation and Robustness Tests

To mitigate overfitting the bot subjects strategies to distributional checks on shuffled data and forward projections of trade sequences. The tooling lives in `src/backtest`.

6.1 Permutation Testing

The `PermutationTester` implements the NeuroTrader methodology by decomposing each bar into overnight gaps and intraday movements, shuffling them independently, and reconstructing synthetic paths. Let $\{G_t\}$ be gaps and $\{I_t\}$ intra-day changes in log space. For each permutation k a synthetic series is rebuilt as

$$\tilde{p}_t^{(k)} = \exp\Big(\tilde{p}_{t-1}^{(k)} + G_{\pi_k(t)} + I_{\sigma_k(t)}\Big)\,. \tag{6.1}$$

Running the strategy backtest on real data yields profit factor Π_{real}; the permutation distribution $\{\Pi^{(k)}\}$ defines the empirical p-value

$$\hat{p} = \frac{1 + \#\{k : \Pi^{(k)} \geq \Pi_{\text{real}}\}}{1 + K}. \tag{6.2}$$

This snippet excerpts the implementation.

```
for i in range(n_permutations):
    fake_data = self.generate_permutation()
    fake_profit = backtest_func(fake_data, **kwargs)
    if fake_profit >= real_profit:
        better_than_real_count += 1
```

6.2 Monte Carlo Equity Cones

The `MonteCarloOptimizer` bootstraps historical trade returns $\{r_j\}_{j=1}^{N}$. For each simulation s a length-T path is generated by resampling with replacement:

$$\tilde{E}_t^{(s)} = E_0 \prod_{u=1}^{t} (1 + r_u^{(s)}), \qquad r_u^{(s)} \sim \text{Empirical}(r_1, \ldots, r_N). \tag{6.3}$$

Risk-of-ruin and quantile equity outcomes follow directly from the simulated terminal distribution. This snippet shows the vectorised implementation.

```
random_returns_matrix = np.random.choice(self.trades, size=(
    num_simulations, num_trades), replace=True)
cum_returns = np.cumprod(1 + random_returns_matrix, axis=1)
equity_curves = self.capital * cum_returns
```

6.3 Synthetic Data Stress

Synthetic price paths generated via `generate_synthetic_data` inject controlled volatility and drift regimes to evaluate strategy behaviour under stylised conditions. The generator draws geometric Brownian motion increments with volatility parameter σ_{syn} and optionally overlays jumps. These series feed directly into the permutation and Monte Carlo pipelines to test stability of conclusions when the data generating process deviates from the historical sample.

Chapter 7

Risk and Performance Metrics

This chapter consolidates the statistical outputs produced by backtests and validation utilities to characterise strategy performance.

7.1 Return Distributions

Given per-period returns R_t, the bot records mean $\bar{R}$, variance $\mathrm{Var}(R)$, skewness, and kurtosis for diagnostic plots. Confidence intervals for the mean assume approximately normal sampling error with standard error $\hat{\sigma}_R/\sqrt{n}$.

7.2 Risk of Ruin

Monte Carlo projections estimate the probability of drawdowns exceeding preset thresholds (e.g., 50%). For threshold ρ and simulated terminal equities $E_T^{(s)}$, risk of ruin is

$$\mathrm{RoR}(\rho) = \frac{1}{S}\sum_{s=1}^{S} \mathbb{1}\{E_T^{(s)} < (1-\rho)E_0\}. \tag{7.1}$$

7.3 Drawdown Analytics

Equity curves E_t yield maximum drawdown as defined in Chapter 5. The duration of drawdowns is also tracked, computed as consecutive streaks where E_t stays below its running maximum.

7.4 Benchmarking

For strategies with reference indices (e.g., the pairs module against SPY) excess return α and information ratio (IR) are reported:

$$\alpha = R_{\text{strat}} - R_{\text{bench}}, \qquad \text{IR} = \frac{\alpha}{\sigma_\alpha}. \tag{7.2}$$

7.5 Capital Utilisation

The `PortfolioManager` maintains utilisation ratios $U_k = E_k/(A_k L_k)$ per strategy k, enabling dashboard alerts when utilisation breaches tolerance levels (typically 80%).

Appendix A

Notation Reference

Symbol	Definition
S_t	Underlying asset price at time t
C_t, P_t	Call and put option prices
K	Option strike price
r	Risk-free interest rate (annualised)
σ	Volatility parameter (annualised)
τ	Time-to-maturity in years
X_t	Equity spread for pairs trading
Y_t	Option spread for vega-neutral pairs
Z_t	Normalised z-score of a spread
$\hat{\beta}$	Hedge ratio derived from cointegration regression
κ_t	Vega hedge ratio between option legs
M_s	Momentum score for sector s
E_t	Strategy equity curve value
Π_j	Profit per individual trade j
R_t	Periodic return
$\Phi(\cdot)$	Standard normal cumulative distribution function
$\phi(\cdot)$	Standard normal probability density function

Table A.1: Key notation used throughout the document.